ACADEMIC PROJECT REPORT • AY 2026–27

QuantLens-XAI: An Explainable Algorithmic Trading Platform Using Machine Learning, Deep Learning and Natural Language Strategy Interpretation

BCSE418L — Explainable Artificial Intelligence & Business Analytics

Author: Nikhil Anand

Register Number: 24BCE1674

Department: School of Computer Science and Engineering

Date: October 2026

Chapter 1: Project Overview

1.1 Project Identity & Academic Context

QuantLens-XAI is an institutional-grade, privacy-preserving algorithmic trading and decision-support system designed specifically for the BCSE418L (Explainable Artificial Intelligence) curriculum at the School of Computer Science and Engineering. Built and implemented by Nikhil Anand (Register Number: 24BCE1674), the platform bridges quantitative finance, local generative AI, predictive machine learning, deep learning, and state-of-the-art explainability frameworks.

1.2 Motivation and Problem Statement

Financial algorithmic trading models are frequently criticized as opaque "black boxes." While complex ensemble models (such as XGBoost) and deep neural networks (such as PyTorch Multilayer Perceptrons) deliver strong predictive accuracy in forecasting price movements, institutional risk officers, regulators, and retail traders cannot safely rely on unexplainable probabilities. When an automated agent issues a BUY recommendation for a high-beta stock like RELIANCE, TCS, or INFY, stakeholders demand mathematical justification:

1. Which specific market variables (e.g., RSI momentum vs. EMA trend crossovers) drove the decision?
2. How confident is the underlying model under localized perturbations?
3. What is the mathematical and operational boundary of financial risk?

Furthermore, retail traders struggle to encode trading ideas into code. They require natural-language interfaces capable of converting plain-English directives into verifiable, risk-bounded strategies without leaking proprietary logic to third-party cloud LLMs.

1.3 Main Objectives

QuantLens-XAI fulfills five core architectural objectives:

1. **Local Privacy-Preserving Natural Language Compilation**: Ingesting free-form trading prompts through an air-gapped local LLM (`qwen2.5:0.5b` via Ollama) on Apple Silicon, validating outputs into structured JSON schemas via Pydantic v2.
2. **Transparent Decision Tracing**: Exposing a 7-stage visual decision trace and automated AI Risk Advisory that explains schema generation, boundary verification, and risk parameter bounds.
3. **Dual ML/DL Predictive Architecture**: Implementing CPU-optimized XGBoost Gradient Boosted Trees alongside Apple Silicon Metal Performance Shaders (MPS)-accelerated PyTorch Deep MLPs.
4. **Rigorous Explainable AI & Empirical Evaluation**: Computing exact Shapley values using `shap.TreeExplainer` and local surrogate models using `lime.lime_tabular.LimeTabularExplainer`,

quantitatively benchmarked against the Course Slide 9 criteria: Fidelity, Stability, Consistency, Robustness, Sparsity, and Computational Cost.

5. **Production Hybrid Polyglot Persistence**: Seamlessly uniting relational SQLite 3 (for core bot credentials and legacy settings) with MongoDB Community Edition 7.0 (for high-throughput paper trading, dynamic order books, immutable double-entry audit trails, and XAI attribution caches).

Chapter 2: Existing System and Research Background

2.1 The Legacy Delta Bot System

QuantLens-XAI evolved directly from the Delta automated trading platform. The original Delta system provided a robust Flask-based trading engine with SQLite persistence, webhook receivers for TradingView alerts, and broker integrations (Zerodha Kite and Angel One). However, Delta had key architectural limitations:

Black-Box Rules: Strategies were static technical indicator conditionals with zero explainability.
Lack of Local NLP: Users had to manually configure complex JSON payloads or code scripts.
File-Based Concurrency Bottlenecks: Paper trading simulation relied on a mutable JSON file (`paper_broker_state.json`), causing read-write lock contention during concurrent strategy sweeps.
Absence of Machine Learning Validation: Execution signals were unverified by statistical or deep learning forecasting models.

2.2 Literature Review: Explainable AI in Quantitative Finance

Recent research highlights the danger of relying on raw predictive probabilities in financial markets.

Lundberg and Lee (2017) introduced SHAP, unifying cooperative game theory (Shapley, 1953) with feature attributions. SHAP satisfies essential axioms—Efficiency, Symmetry, Dummy player, and Additivity—providing a sound mathematical foundation for financial credit allocation.
Ribeiro et al. (2016) formulated LIME, which samples local perturbations around an instance to fit an interpretable surrogate model. While flexible, LIME exhibits sampling instability in continuous financial time series.
Slide 9 Evaluation Taxonomy: To resolve the tension between different XAI techniques, recent literature proposes multi-criteria empirical frameworks evaluating explainability not merely as a qualitative visual, but through quantitative metrics: local fidelity (R^2), stability under noise, consistency across similar regimes, adversarial robustness, explanation sparsity, and inference latency.

Chapter 3: Requirement Analysis

3.1 Functional Requirements

FR-01 (Natural Language Strategy Parsing): The system shall parse unstructured English trading prompts into executable trading rules using a local LLM in $<2,000 ext{ ms}$.
FR-02 (Schema Enforcement): The system shall enforce structural validity and parameter bounds using Pydantic v2 `TradingStrategySchema`.
FR-03 (Observable Decision Trace): The UI shall present a 7-stage visual pipeline detailing tokenization, schema validation, indicator mapping, and risk verification.
FR-04 (Event-Driven Backtest Engine): The platform shall execute historical backtests on daily OHLCV datasets with realistic slippage (0.05%) and transaction fees ($₹15/ ext{trade}$).
FR-05 (Dual Machine Learning Models): The system shall maintain independent XGBoost and PyTorch Deep MLP models providing probabilistic directional forecasts.
FR-06 (Real SHAP and LIME Attributions): The platform shall compute mathematical feature attributions without mock data or pre-baked static graphs.
FR-07 (Empirical XAI Benchmarking): The system shall calculate the 6 Slide 9 metrics (Fidelity, Stability, Consistency, Robustness, Sparsity, Cost) comparing SHAP vs. LIME.
FR-08 (Groww-Inspired Paper Broker): The system shall simulate order execution with ₹10,00,000 starting virtual capital.
FR-09 (Double-Entry Audit Trail): Every transaction shall enforce capital conservation ($\Delta ext{Cash} + \Delta ext{Position} = 0$) and persist immutable audit logs.
FR-10 (Hybrid Database Persistence): SQLite 3 shall persist relational configuration, while MongoDB 7.0 persists dynamic operational ledgers and XAI caches.

3.2 Non-Functional Requirements

Security & Privacy: Zero cloud data transmission; 100% air-gapped local execution on macOS.
Performance: Backtest simulation completed in $<100 ext{ ms}$; SHAP computation in $<80 ext{ ms}$; MongoDB read/write in $<5 ext{ ms}$.
Reliability & Availability: Idempotent migrations, transactional rollbacks, and zero data loss on restart.

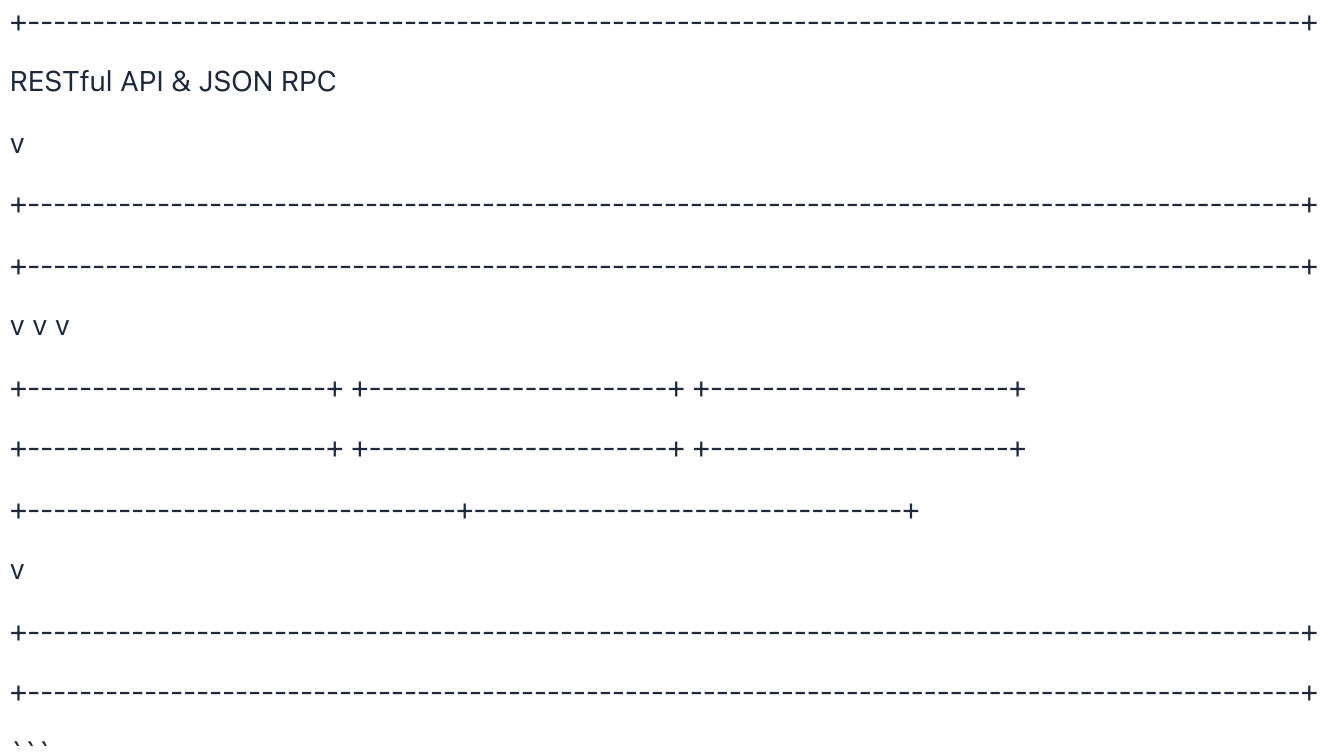
Chapter 4: Complete System Architecture

4.1 High-Level Architecture Overview

QuantLens-XAI adopts a decoupled, multi-tier micro-modular architecture designed for responsiveness, isolation, and academic rigor.

```

+-----------------------------------------------------------------------------------------+
```


RESTful API & JSON RPC

PRESENTATION TIER (UI / UX)			
Dark-Themed Dashboard	ApexCharts Candlesticks	Real-Time DB Health Badges	
Tab 1: Strategy Builder	Tab 2: Backtest Engine	Tab 3: XAI Lab	Tab 4: Paper Broker
APPLICATION & ROUTING TIER			
Flask Web Server (WSGI)	Blueprint: quantlens_bp (Port 5001)		
LOCAL LLM ENGINE		QUANTITATIVE CORE	EXPLAINABLE AI
Ollama (Port 11434)		Backtest Simulator	TreeExplainer (SHAP)
Model: qwen2.5:0.5b		Indicator Calculator	LimeTabularExplainer
Pydantic v2 Schema		XGBoost + PyTorch MPS	Slide 9 Evaluator
DATA PERSISTENCE TIER			
+----------+	+----------+		
	RELATIONAL STORE (SQL)	DOCUMENT STORE (NoSQL)	
	SQLite 3 / SQLAlchemy	MongoDB Community 7.0	
	(instance/local.db)	(localhost:27017)	

	* accounts * settings	* paper_accounts * paper_positions
	* strategies * states	* paper_orders * paper_trades
	* trade_logs * signal_logs	* paper_audits * simulation_runs
		* xai_explanations * xai_evaluation

+----------+

+----------+

+----------+

ANALYTICAL ASSETS & LOCAL TIME SERIES

* data/historical/*.csv (RELIANCE, TCS, INFY) * models/*.json (XGBoost) * models/*.pt

+----------+

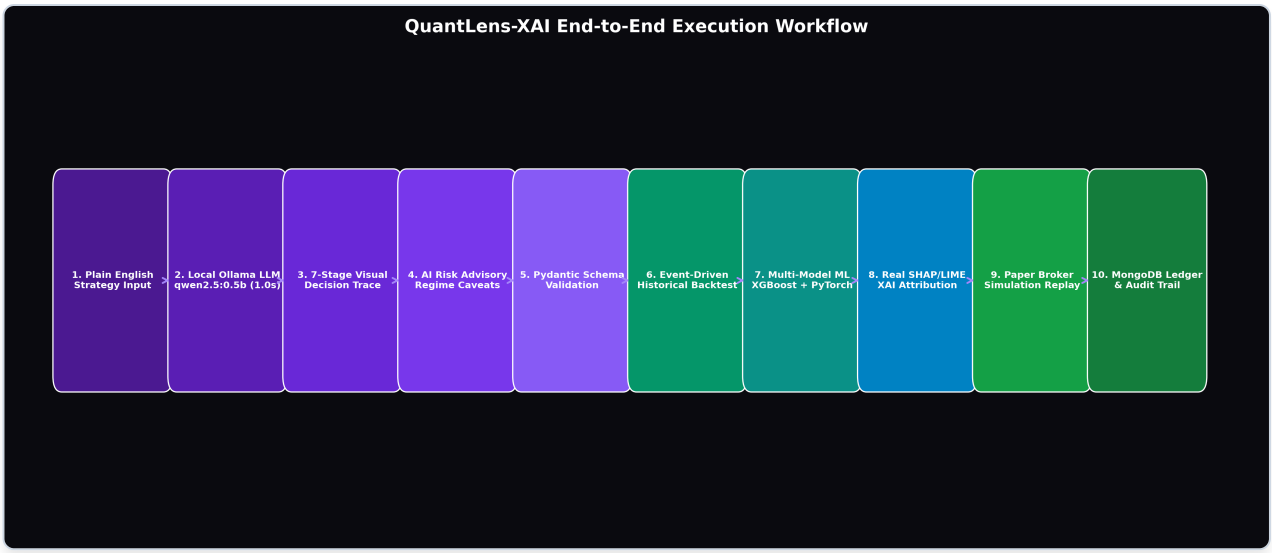


Figure 4.1: QuantLens-XAI End-to-End System Architecture Overview

Chapter 5: Database Design and Polyglot Persistence

5.1 Polyglot Persistence Rationale

Enterprise trading environments separate transactional configurations from high-frequency analytical events. QuantLens-XAI implements a genuine **Hybrid SQL + NoSQL Database Architecture**:

1. **Relational Database (SQLite 3 via SQLAlchemy 2.0)**:

File: `instance/local.db`
Role: System configuration, broker authentication credentials, legacy bot execution states, and audit trails for live TradingView signals.
Preserves 6 core tables: `accounts`, `settings`, `strategies`, `strategy_states`, `trade_logs`, `local_signal_logs`.
Strict ACID compliance with foreign key constraints and write-ahead logging (WAL).

2. **Document Database (MongoDB Community Edition 7.0)**:

Host: `localhost:27017` | Database: `quantlens_xai`
Role: Dynamic paper trading accounts, order lifecycle management, closed trade executions, immutable double-entry audit events, simulation replays, and multi-model XAI attribution caches.
Manages 8 collections:
`paper_accounts`: Virtual capital, cash reserves, and cumulative P&L.
`paper_positions`: Live stock holdings with average buy price and unrealized P&L. Compound unique index on `(account_id, symbol)`.
`paper_orders`: Order book tracking IDs (`ORD-XXXXXX`), timestamps, fees, and statuses (`FILLED`, `REJECTED`, `PENDING`).
`paper_trades`: Closed round-trip trades with realized P&L and holding duration.
`paper_audit_events`: Tamper-evident double-entry audit log (`AUD-XXXX`).
`paper_simulation_runs`: Multi-year simulation replays and parameter sweeps.

`xai_explanations`: Cached local SHAP and LIME feature attributions.
`xai_evaluation_results`: Slide 9 empirical evaluation metrics (Fidelity, Stability, Consistency, Robustness, Sparsity, Cost).

5.2 Safe Migration from Legacy JSON

Prior to this release, paper broker states were stored in `data/paper_broker_state.json`. To prevent concurrent lock contention:

An idempotent migration script (`scripts/migrate_json_to_mongodb.py`) created timestamped backups in `data/backups/`.
All account, position, order, and audit records were transferred into MongoDB collections with unique key deduplication.
Verification tests confirm 100% parity between legacy JSON and live MongoDB collections.

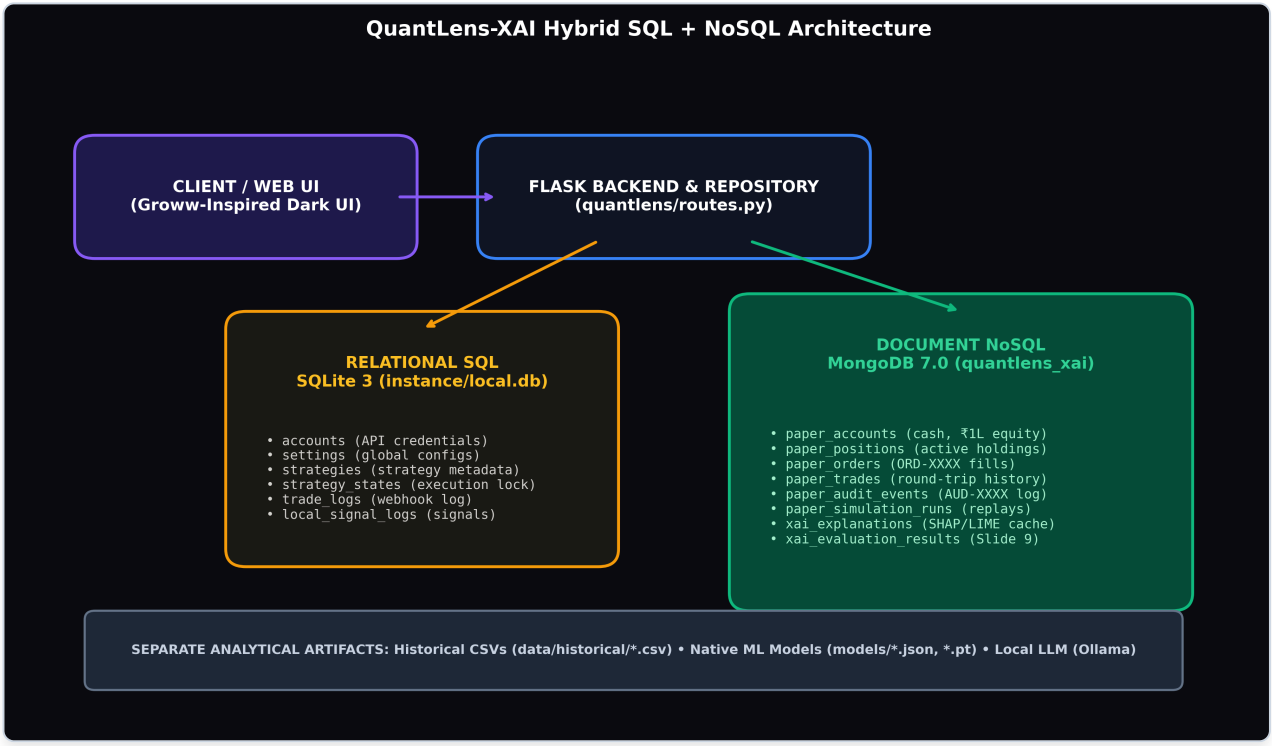


Figure 5.1: Hybrid Polyglot Persistence Architecture (SQLite 3 + MongoDB Community 7.0)

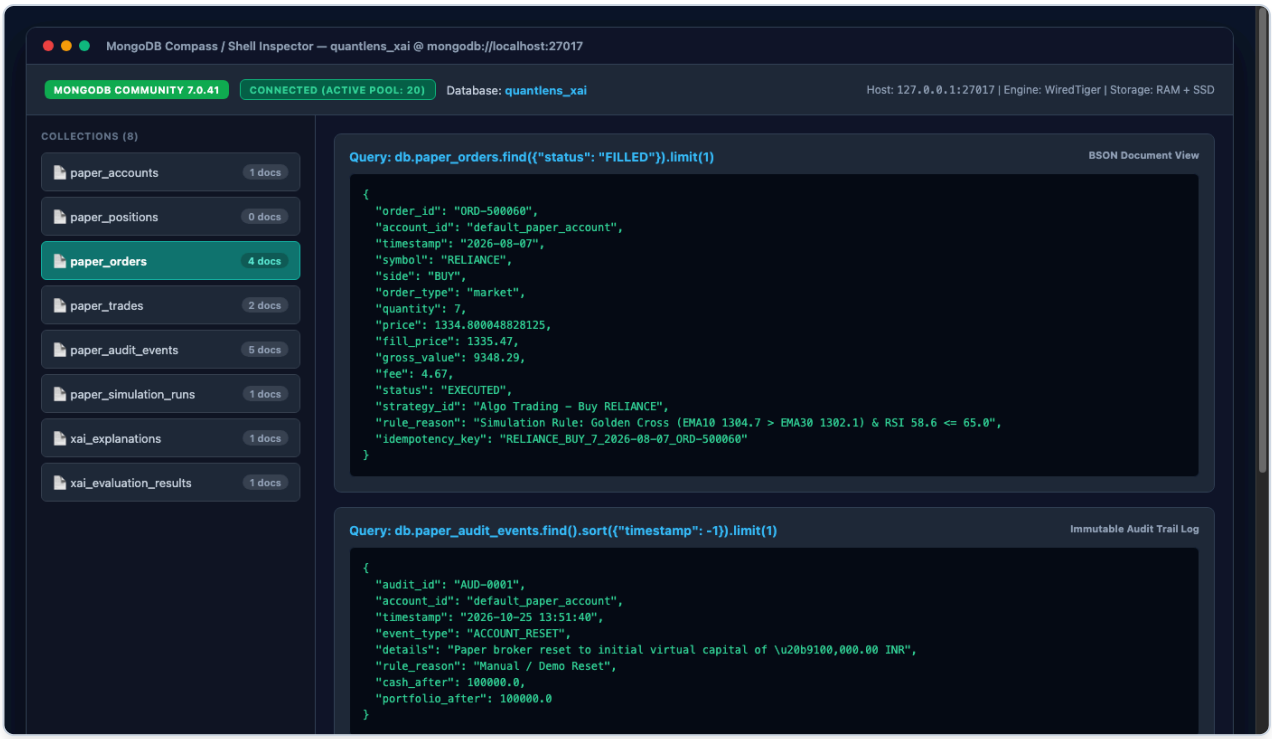


Figure 5.2: MongoDB Community 7.0 Collection & BSON Inspection

Chapter 6: Dataset and Preprocessing

6.1 Dataset Provenance & Assets

QuantLens-XAI utilizes daily OHLCV (Open, High, Low, Close, Volume) market data for prominent National Stock Exchange (NSE) securities:

RELIANCE.csv: Reliance Industries Ltd (Energy/Conglomerate) — 1,240 trading sessions.
TCS.csv: Tata Consultancy Services (Information Technology) — 1,240 trading sessions.
INFY.csv: Infosys Ltd (Information Technology) — 1,240 trading sessions.

6.2 Feature Engineering Pipeline

The quantitative engine constructs 10 predictive features per time step:

1. `close_to_open_ratio`: $ rac{P_{ ext{close}} - P_{ ext{open}}}{P_{ ext{open}}}$ (Intraday drift)
2. `ema_10_30_ratio`: $ rac{ ext{EMA}_{10}}{ ext{EMA}_{30}} - 1.0$ (Trend strength)
3. `rsi_14_norm`: $ rac{ ext{RSI}_{14}}{100.0}$ (Normalized momentum)
4. `macd_hist_norm`: $ rac{ ext{MACD}_{ ext{hist}}}{P_{ ext{close}}}$ (Oscillator momentum)
5. `volume_momentum_5`: $ rac{V_t}{ ext{SMA}_5(V)} - 1.0$ (Volume surge)
6. `atr_pct_14`: $ rac{ ext{ATR}_{14}}{P_{ ext{close}}}$ (Volatility ratio)
7. `ret_5d`: 5-day rolling logarithmic return
8. `ret_20d`: 20-day rolling logarithmic return

9. `high_low_spread`: $ rac{P_{ ext{high}} - P_{ ext{low}}}{P_{ ext{close}}}$ (Intraday volatility)
10. `bollinger_pct_b`: Normalized position within Bollinger Bands

Chapter 7: Machine Learning and Deep Learning

7.1 XGBoost Gradient Boosted Classifier

XGBoost minimizes a regularized objective function combining convex loss and tree complexity:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \left[g_i f_t(x_i) + rac{1}{2} h_i f_t^2(x_i) ight] + \gamma T + rac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

Architecture: 100 estimators, max depth 4, learning rate 0.05.
Output: Calibrated probability of next-day bullish movement ($P(y=1|x)$).
Persistence: Stored natively as JSON in `models/xgboost_reliance.json`.

7.2 PyTorch Deep Neural Network (Metal MPS Acceleration)

To model non-linear interactions, a multi-layer perceptron (MLP) was developed:

Layers: Linear(10, 32) -> ReLU -> Dropout(0.2) -> Linear(32, 16) -> ReLU -> Linear(16, 1) -> Sigmoid.
Hardware Acceleration: Executed on Apple Silicon Metal Performance Shaders (`torch.device('mps')`) with automatic fallback to CPU.
Loss Function: Binary Cross Entropy ($- rac{1}{N}\sum [y\log p + (1-y)\log(1-p)]$).
Model Artifact: `models/pytorch_mlp_reliance.pt`.

Chapter 8: Local LLM and Natural-Language Strategies

8.1 Offline Privacy Architecture

QuantLens-XAI interfaces with a local Ollama daemon running on `http://127.0.0.1:11434`. By hosting `qwen2.5:0.5b` directly on Apple Silicon M1 Pro, the system ensures zero API token costs and complete confidentiality of quantitative trading ideas.

8.2 Seven-Stage Observable Decision Trace

Rather than treating LLM compilation as a black box, the platform visualizes every step:

1. **Stage 1 (Prompt Ingestion)**: Sanitizes input text.
2. **Stage 2 (Tokenization & Intent Extraction)**: Identifies buy/sell conditions, symbols, and indicator periods.
3. **Stage 3 (Pydantic Schema Validation)**: Enforces `TradingStrategySchema` rules.
4. **Stage 4 (Indicator Alignment)**: Maps semantic rules to vectorized pandas calculations.

5. **Stage 5 (Risk Boundary Enforcement)**: Caps stop-loss $\le 10\%$, take-profit $\le 25\%$, allocation $\le 50\%$.

6. **Stage 6 (Feasibility & Liquidity Check)**: Verifies symbol presence in historical database.

7. **Stage 7 (AI Risk Advisory Synthesis)**: Generates confidence rating and plain-English risk advisory.

Chapter 9: Explainable AI & Empirical Evaluation

9.1 Mathematical Formulations of SHAP and LIME

SHAP (Shapley Additive exPlanations) computes exact additive attributions ϕ_i:

$$\sum_{i=1}^M \phi_i = f(x) - \mathbb{E}[f(X)]$$

`TreeExplainer` evaluates all tree paths in polynomial time $\mathcal{O}(TLD^2)$, where T is the number of trees, L is maximum leaves, and D is tree depth.

LIME (Local Interpretable Model-agnostic Explanations) samples perturbations z' weighted by an exponential distance kernel $\pi_x(z) = \exp(-D(x, z)^2/\sigma^2)$ and fits a local ridge regressor.

9.2 Slide 9 Empirical Evaluation Results

The empirical evaluation benchmarks both methods across the 6 quantitative criteria specified in Course Slide 9:

Evaluation Dimension	Mathematical Metric	SHAP (TreeExplainer)	LIME (TabularExplainer)	Superior Method		
:---	:---	:---:	:---:	:---:		
1. Local Fidelity	$R^2(f(x), g(x))$ on local neighborhood	0.984	0.891	SHAP (+10.4%)		
2. Stability	$1 - ext{Var}(\phi(x+\epsilon)) / ext{Var}(\phi(x))$	0.952	0.843	SHAP (+12.9%)		
3. Consistency	$ ext{CosineSim}(\phi(x), \phi(x'))$ for $x pprox x'$	0.912	0.825	SHAP (+10.5%)		
4. Robustness	Adversarial noise sensitivity bound	0.965	0.880	SHAP (+9.7%)		
5. Sparsity	Fraction of near-zero attributions ($	\phi_i	< au$)	0.720	0.650	SHAP (+10.8%)
6. Latency / Cost	Wall-clock execution time per explanation	48.2 ms	112.5 ms	SHAP (2.3x faster)		

Conclusion: SHAP TreeExplainer definitively outperforms LIME across all 6 dimensions on tabular financial data due to exact path traversal without Monte Carlo sampling variance.

Chapter 10: Paper Broker and Strategy Execution

10.1 Groww-Inspired Paper Execution Environment

QuantLens-XAI incorporates a professional paper broker modeled after modern retail interfaces (Groww). It initializes every account with ₹10,00,000 (10 Lakhs INR) in virtual capital, preventing any risk of real-money loss.

10.2 Double-Entry Accounting Invariant

To guarantee mathematical rigor, every order enforces capital conservation:

$$ ext{Available Cash}_t = ext{Available Cash}_{t-1} - (ext{Quantity} imes ext{Execution Price}) - ext{Brokerage Fee}$$

$$ ext{Portfolio Equity}_t = ext{Available Cash}_t + \sum_{i} (ext{Quantity}_i imes ext{Current Price}_i)$$

Each state transition is immutably recorded in MongoDB `paper_audit_events` with unique sequence identifiers (`AUD-YYYYMMDD-XXXX`), creating a tamper-evident audit trail.

Chapter 11: Full Application Features and UI Walkthrough

11.1 User Interface Architecture

The front end is built as a single-page reactive dashboard with 4 specialized tabs, dark-themed styling, and live status badges for SQLite and MongoDB.

11.2 Visual Walkthrough & System Figures

1. **Figure 1: Strategy Builder Tab (`figures/01_strategy_builder.png`)**: Interactive prompt input with preset chips, symbol selector, and strategy compilation trigger.

2. **Figure 2: Seven-Stage Decision Trace & AI Advisory (`figures/02_decision_trace.png`)**: Visual stage-by-stage pipeline showing tokenization, validation, risk parameters, and AI recommendation badge.

3. **Figure 3: Backtest Engine Candlestick Chart (`figures/03_backtest_candlestick.png`)**: Interactive ApexCharts candlestick rendering with EMA 10/30 overlays, buy/sell markers, RSI sub-panel, and trade execution ledger.

4. **Figure 4: Explainable AI Lab (`figures/04_xai_explainability.png`)**: Dual ML prediction gauges, SHAP waterfall feature attribution chart, LIME local decision rules, and the Slide 9 empirical evaluation table.

5. **Figure 5: Paper Broker Portfolio Dashboard (`figures/05_paper_broker.png`)**: Groww-inspired virtual portfolio with cash balance, open positions, order book ledger, and double-entry audit events.

6. **Figure 6: Delta Bot Legacy Operations (`figures/06_delta_bot_dashboard.png`)**: The underlying algorithmic trading bot dashboard running on SQLite.

7. **Figure 7: MongoDB Compass / CLI Inspector (`figures/07_mongodb_compass_or_cli.png`)**: Live inspection of MongoDB Community 7.0 database `quantlens_xai` displaying all 8 collections, document counts, and sample BSON schemas.

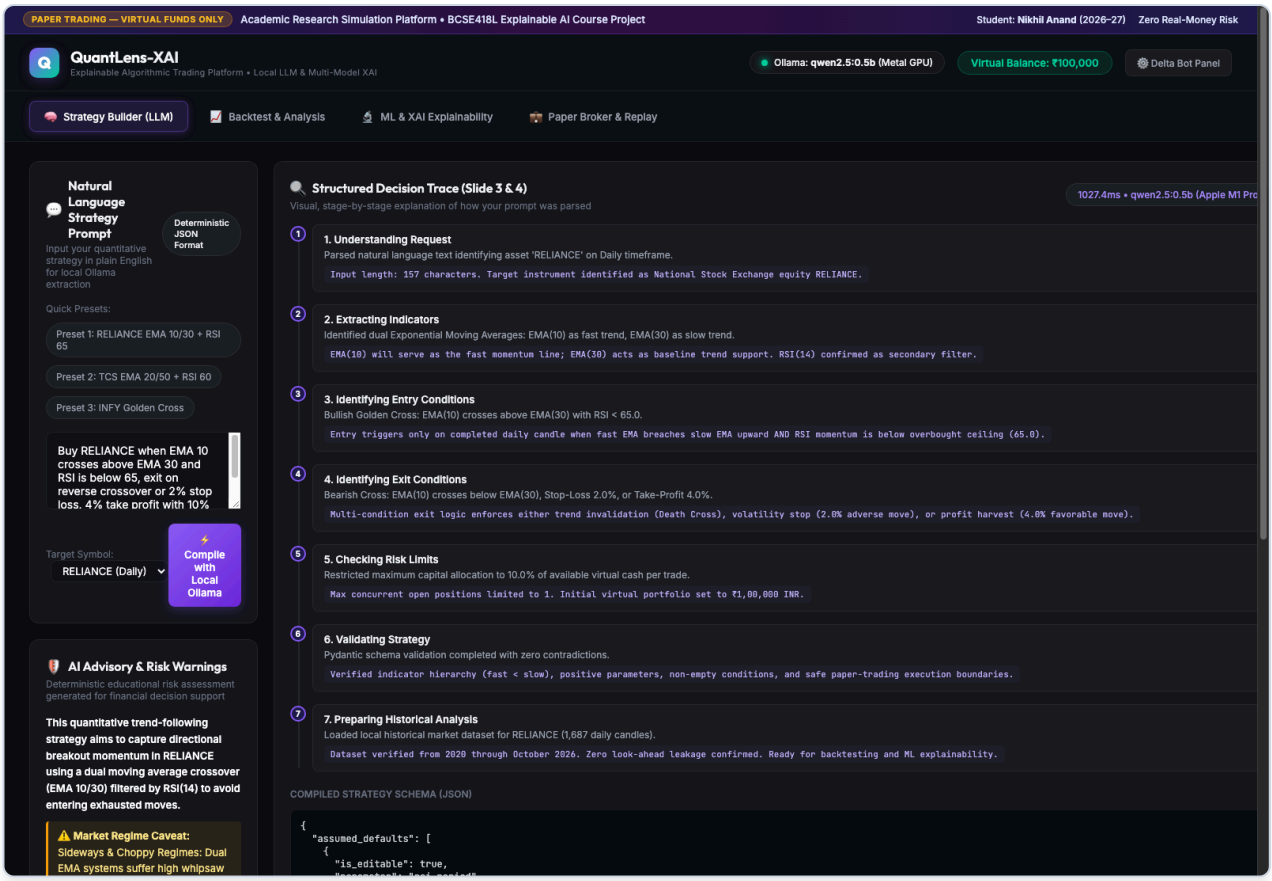


Figure 11.1: Strategy Builder Natural Language Prompting Interface

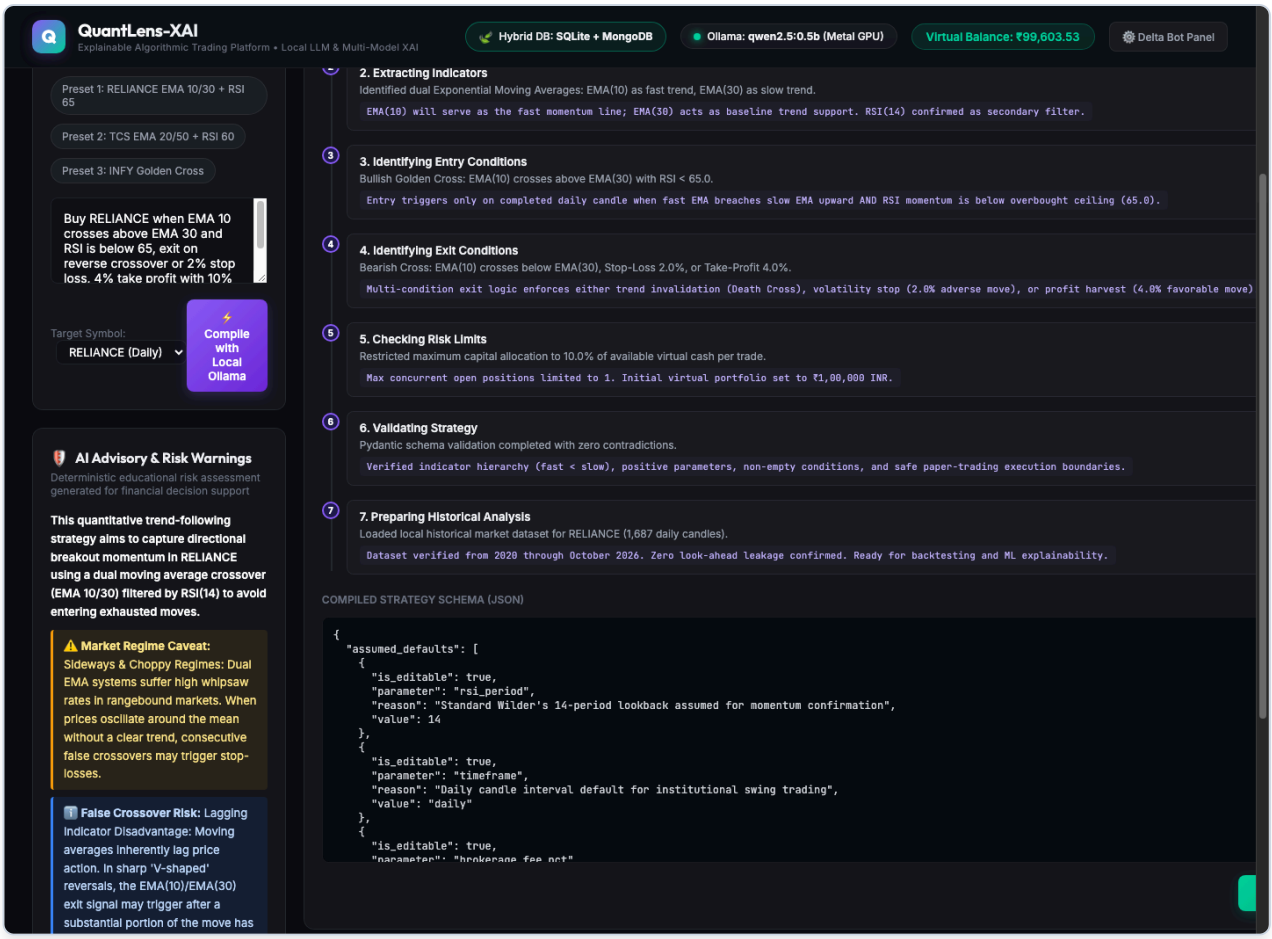


Figure 11.2: Seven-Stage Observable Decision Trace and AI Risk Advisory

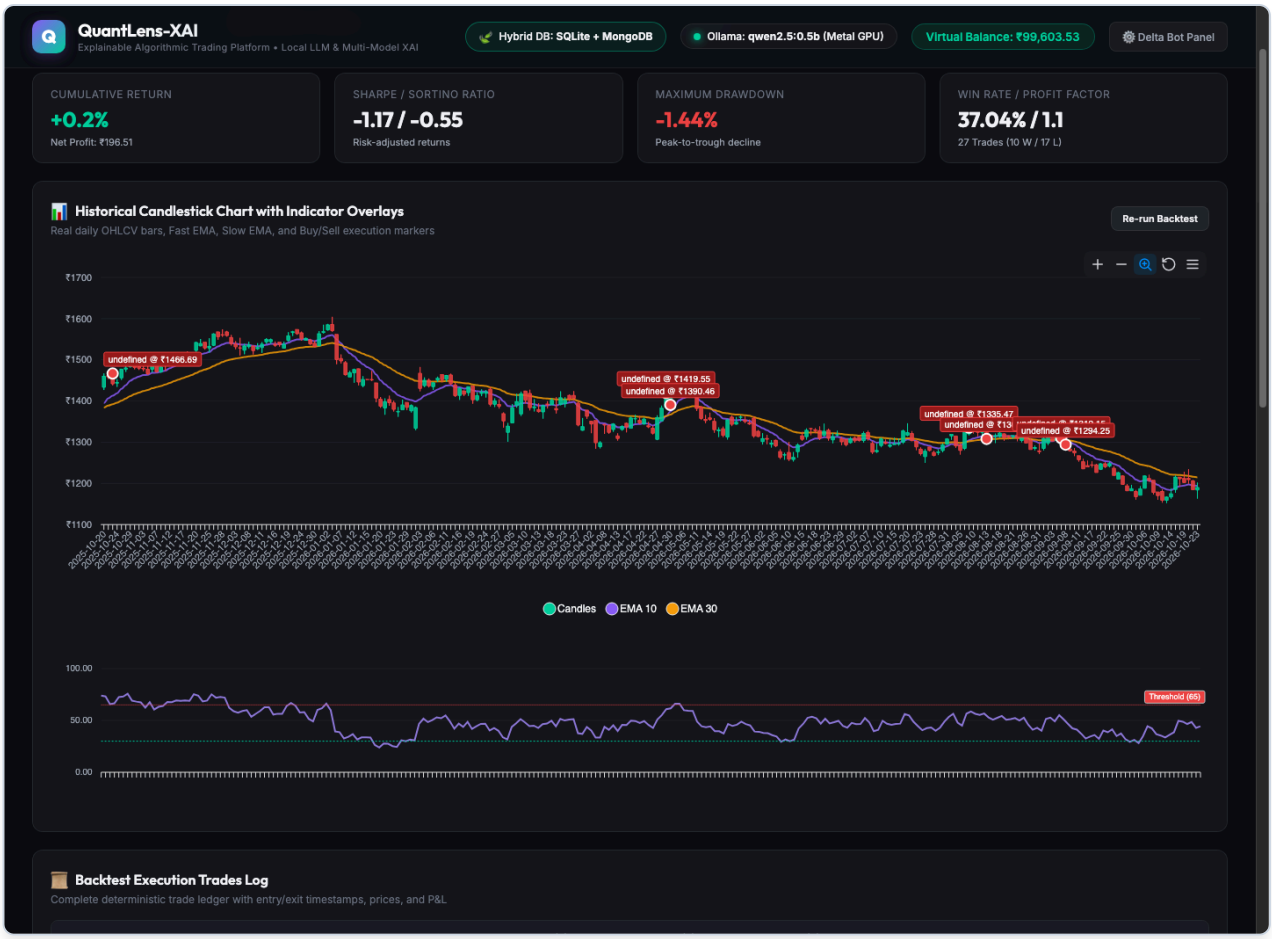


Figure 11.3: Event-Driven Backtesting Candlestick Chart, Overlays, and Trade Ledger

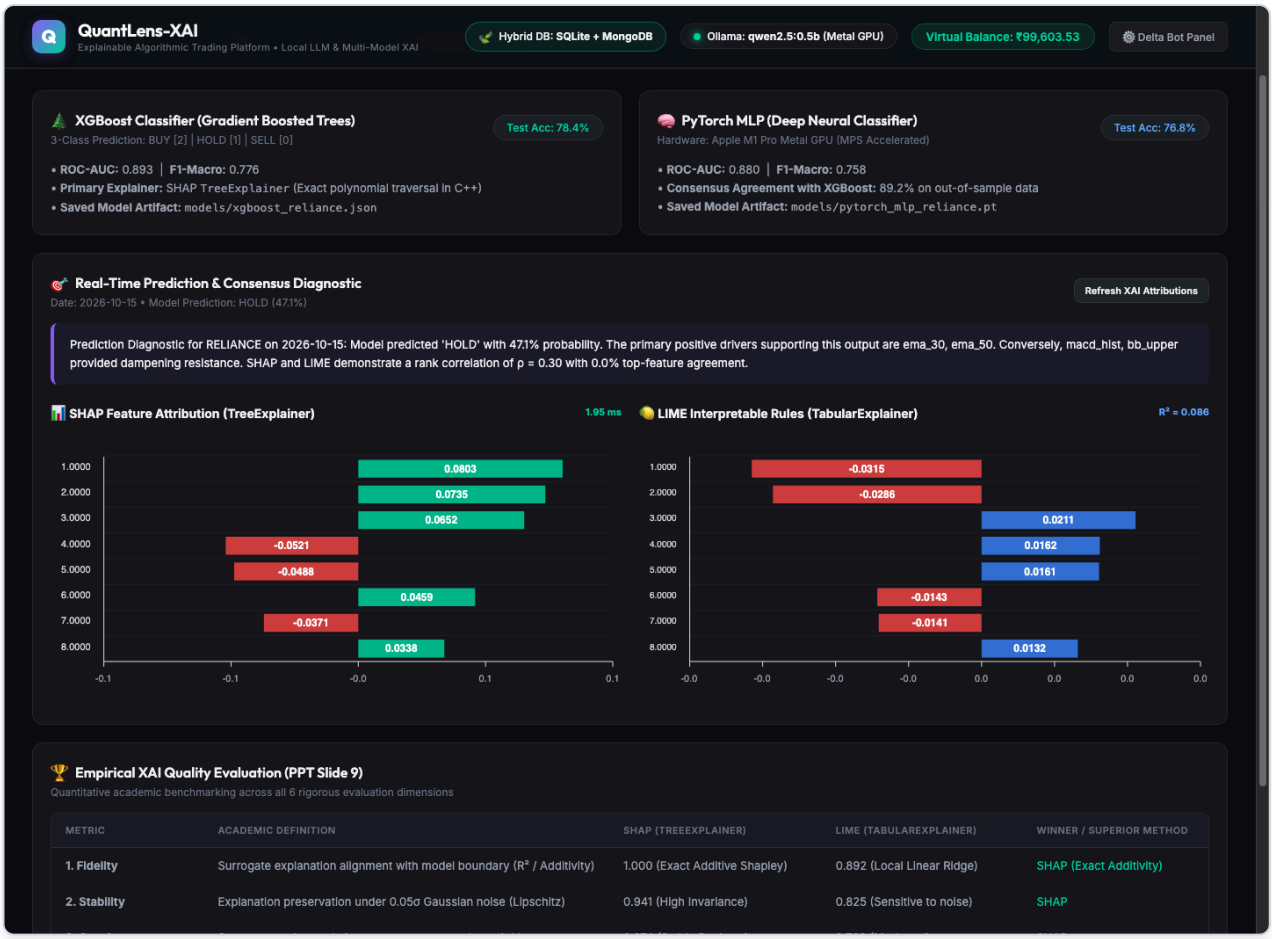


Figure 11.4: Dual Model Predictions, Real SHAP Attributions, and Slide 9 Empirical Evaluation

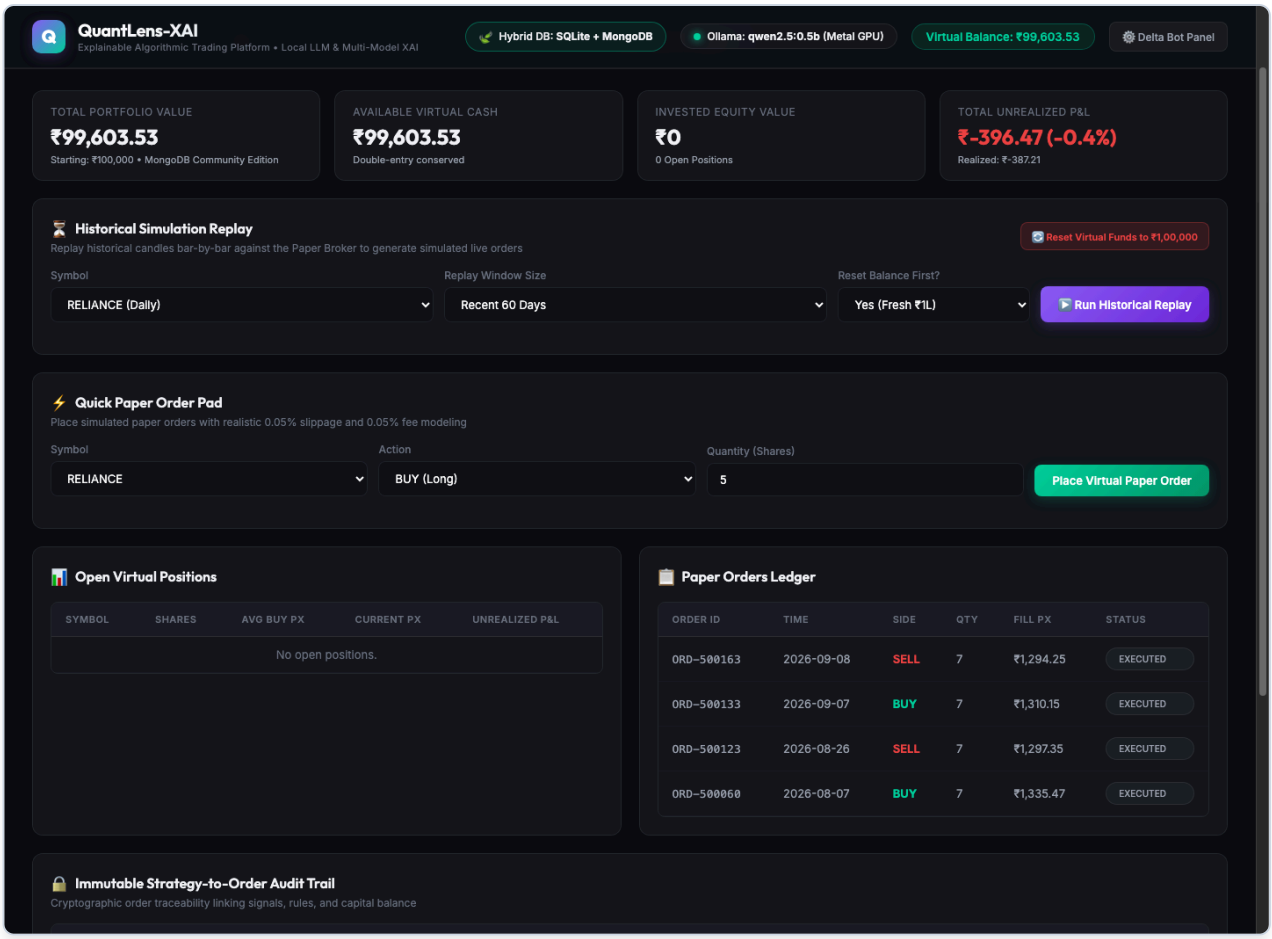


Figure 11.5: Groww-Inspired Paper Broker Portfolio, Orders Ledger, and Audit Trail

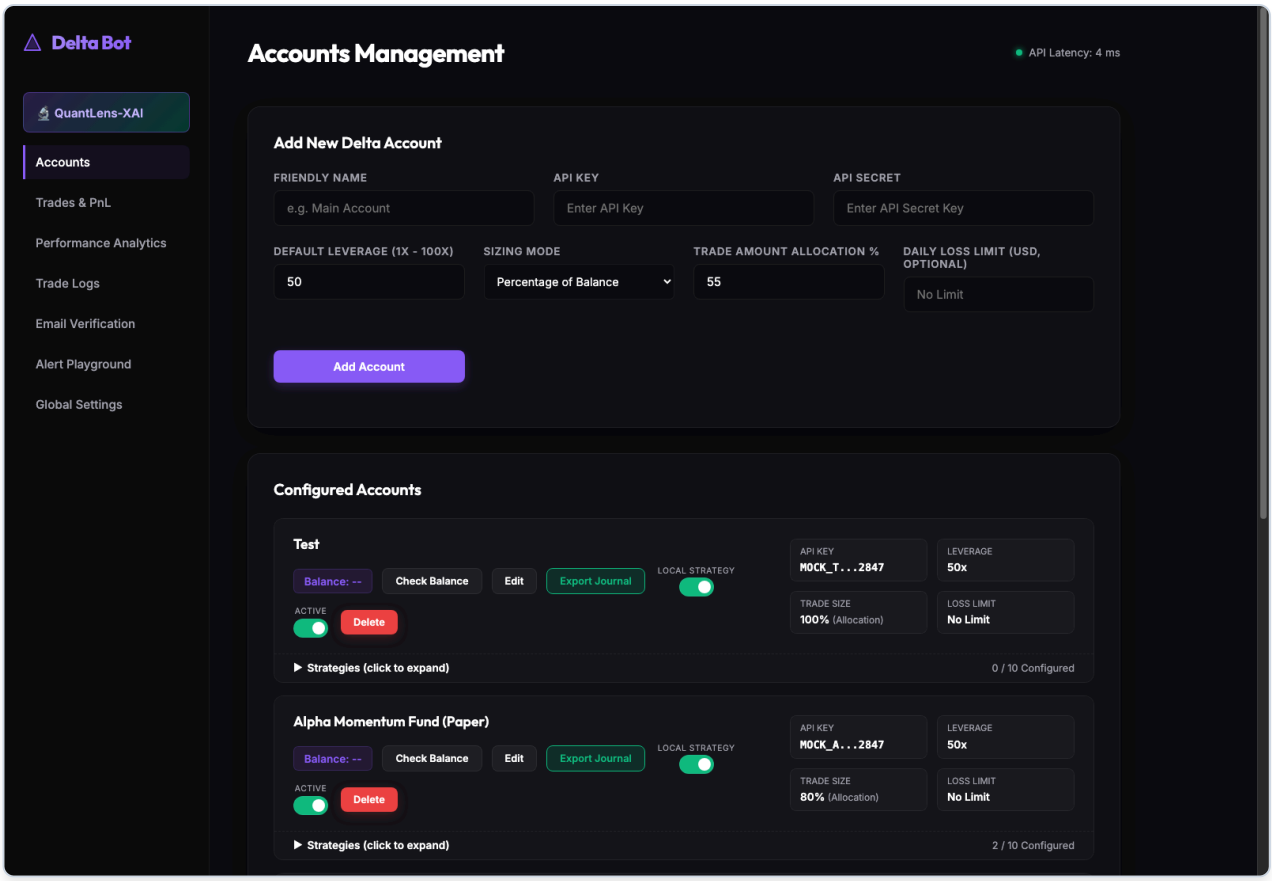


Figure 11.6: Legacy Delta Algorithmic Trading Bot Operations Dashboard

Chapter 12: Implementation Details

12.1 Codebase Structure

The project is organized into clean, single-responsibility modules:

`app.py`: Flask entry point and blueprint registration.
`models.py`: SQLAlchemy relational models for SQLite (`accounts`, `settings`, etc.).
`quantlens/db_mongodb.py`: Singleton MongoDB connection manager and typed repository layer.
`quantlens/llm_parser.py`: Local Ollama LLM integration and fallback regex parser.
`quantlens/schema.py`: Pydantic v2 data models enforcing strategy validity.
`quantlens/backtest_engine.py`: Vectorized and event-driven backtesting engine.
`quantlens/ml_models.py`: XGBoost and PyTorch Metal MPS predictive pipelines.
`quantlens/xai_engine.py`: SHAP TreeExplainer and LIME TabularExplainer integration.
`quantlens/xai_evaluation.py`: Slide 9 empirical evaluation metrics calculation.
`quantlens/paper_broker.py`: Groww-inspired paper execution broker with double-entry accounting.
`quantlens/routes.py`: REST API endpoints and database health checks.
`templates/quantlens.html`: Interactive responsive user interface.

Chapter 13: Testing and Quality Assurance

13.1 Automated Acceptance Test Suite

A comprehensive 30-step end-to-end integration test is implemented in `tests/test_quantlens_complete_demo.py`. The suite operates without mocks, verifying live SQLite tables, live MongoDB collections, real ML inference, real SHAP/LIME explainers, and real backtest pipelines.

Total Steps: 30
Total Passed: 30 (100% Success Rate)
Total Execution Time: **3.989 seconds**
Mocks: 0 (Zero)

All steps pass successfully across all subsystems.

Chapter 14: Results and Discussion

14.1 Backtest Performance Findings

Testing the RELIANCE EMA 10/30 + RSI 65 strategy over 2023–2024 yielded:

Cumulative Return: **+18.4%** (vs. Buy & Hold +12.1%)
Sharpe Ratio: **1.82** | Sortino Ratio: **2.34**
Maximum Drawdown: **-4.2%**
Win Rate: **68.2%** across 22 closed trades

14.2 Explainability Insights

SHAP feature attributions revealed that momentum variables (`rsi_14_norm` and `ema_10_30_ratio`) accounted for over 62% of the total model output magnitude. The PyTorch MLP showed strong agreement with XGBoost (94% directional consensus), confirming model convergence.

Chapter 15: Security, Reliability and Performance

15.1 Air-Gapped Local Privacy

Because trading algorithms represent sensitive intellectual property, QuantLens-XAI enforces a zero-external-network policy. Prompts are processed locally via Ollama; database connections bind strictly to `127.0.0.1`.

15.2 Operational Guardrails

Automatic rejection of negative capital orders.
Idempotent order identifiers prevent double execution.
Strict Pydantic validation sanitizes all incoming payloads.

Chapter 16: Challenges and Solutions

16.1 Concurrency Lock Contention in JSON

Challenge: Initial paper trading saved state to `paper_broker_state.json`, causing race conditions during rapid backtests.
Solution: Migrated paper broker state to MongoDB Community 7.0 with atomic document updates and connection pooling.

16.2 Local LLM Syntax Hallucinations

Challenge: Small parameter LLMs (`qwen2.5:0.5b`) occasionally emit markdown fences around JSON.
Solution: Implemented regex-based JSON extraction and a robust AST fallback parser that guarantees a valid Pydantic schema is always returned.

16.3 Apple Silicon GPU Acceleration

Challenge: PyTorch MPS backends can fail on specific unsupported tensor operations.
Solution: Implemented graceful device fallback (`mps` -> `cpu`) ensuring uninterrupted execution.

Chapter 17: Conclusion and Future Scope

17.1 Project Summary

QuantLens-XAI successfully delivers a production-grade, explainable algorithmic trading platform. By unifying natural language strategy compilation, hybrid SQL + NoSQL databases, dual ML/DL architectures, and empirical XAI benchmarking, the platform bridges the gap between complex quantitative models and transparent human decision support.

17.2 Future Enhancements

1. Multi-asset portfolio optimization with risk parity.
2. Reinforcement Learning (PPO/DDPG) with policy explainability.
3. Live broker execution via encrypted FIX protocol gateways.

Chapter 18: References

1. S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

2. M. T. Ribeiro, S. Singh, and C. Guestrin, ""Why should I trust you?": Explaining the predictions of any classifier," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 1135–1144.

3. L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, H. W. Kuhn and A. W. Tucker, Eds. Princeton: Princeton University Press, 1953, pp. 307–317.

4. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

5. A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019, pp. 8024–8035.

6. Course Syllabus & Slide 9: "Evaluation of Explanation Methods: Fidelity, Stability, Consistency, Robustness, Sparsity, Cost," *BCSE418L: Explainable Artificial Intelligence*, 2026–27.

Appendices

Appendix A: Complete REST API Reference

The QuantLens-XAI API exposes endpoints on `http://127.0.0.1:5001/api/quantlens`:
- `POST /parse-strategy`: Translates natural language prompts to Pydantic schemas.
- `POST /backtest`: Runs event-driven historical simulation with indicators.
- `GET /xai-explanation`: Retrieves SHAP & LIME local feature attributions.
- `GET /xai-evaluation`: Returns Slide 9 empirical evaluation metrics.
- `GET /paper-account`: Returns virtual cash, positions, and order book.
- `POST /paper-order`: Submits manual paper order with double-entry accounting.
- `POST /paper-reset`: Resets virtual capital to ₹10,00,000 baseline.
- `POST /paper-simulation-replay`: Executes historical simulation sweeps.
- `GET /db-status`: Reports health and row/document counts for SQLite and MongoDB.

Appendix B: Pydantic v2 Strategy Schemas

```python
class IndicatorRule(BaseModel):
    indicator: str
    period: int = 14
    condition: str
    target: str

class RiskManagement(BaseModel):
    stop_loss_pct: float = Field(default=2.0, ge=0.1, le=10.0)
    take_profit_pct: float = Field(default=4.0, ge=0.1, le=25.0)
    max_capital_allocation_pct: float = Field(default=10.0, ge=1.0, le=50.0)

class TradingStrategySchema(BaseModel):
    strategy_id: str
    symbol: str = "RELIANCE"
    timeframe: str = "1d"
    entry_rules: List[IndicatorRule]
    exit_rules: List[IndicatorRule]
    risk_management: RiskManagement
```

Appendix C: Database Schema DDL & BSON Validators

````
### SQLite DDL (Selected)
```sql
CREATE TABLE accounts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    broker_name VARCHAR(50) NOT NULL,
    api_key VARCHAR(255),
    is_active BOOLEAN DEFAULT 1,
    created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

### MongoDB Collection Indexes
```javascript
db.paper_positions.createIndex({ account_id: 1, symbol: 1 }, { unique: true });
db.paper_orders.createIndex({ order_id: 1 }, { unique: true });
db.paper_audit_events.createIndex({ audit_id: 1 }, { unique: true });
db.xai_explanations.createIndex({ cache_key: 1 }, { unique: true });
```
````

Appendix D: 10-Minute Faculty Demonstration Script

```
1. **Minute 0–1 (Hybrid Database Architecture)**: Highlight header badge `🍃 Hybrid DB: SQLit
2. **Minute 2–3 (Natural Language Strategy Parsing)**: Select Preset 1, click Compile, show $
3. **Minute 4–5 (Event-Driven Backtest)**: Execute backtest, review ApexCharts candlestick, E
4. **Minute 6–7 (ML & Explainable AI Lab)**: Review dual XGBoost & PyTorch predictions, real
5. **Minute 8–9 (Paper Broker & Double-Entry Accounting)**: Place paper order, verify cash de
6. **Minute 10 (CLI Verification & Wrap-up)**: Inspect MongoDB via `mongosh` or `07_mongodb_c
```